RescueOps

Problem Statement: Smart Rescue Decision Agent

1. Background

During a disaster, rescue teams receive several calls for help at the same time. Each call (mission) has a different severity, distance and deadline, roads may be blocked, the weather may be bad, and the team has only a limited amount of fuel, medical kits and personnel. A wrong decision, such as sending a team on a blocked road or without enough fuel, can cost lives.

2. Objective

Build an **AI agent** (in Python, using LangGraph and LangChain) that reads a rescue scenario and returns one **safe and smart decision**: which mission to send help to, and how, or whether to wait, reject or ask for resupply. The competition has **5 rounds** of increasing difficulty (100 points in total). A single `agent.py` must handle all five rounds, and each round includes everything from the earlier ones.

Round	Title	Points	New challenge
1	Basic Rescue	10	One mission; check fuel and deadline
2	Blocked Routes	15	Blocked roads and alternate routes
3	Limited Resources	20	Fuel, kits and personnel are limited; resupply
4	Critical Priorities	25	Many missions with tight deadlines; severity vs feasibility
5	Adaptive RescueOps	30	Weather changes and free-text updates; decide again

3. Input: the Scenario (case)

```
case_id, round, stage
weather ("clear" | "rain" | "storm")
resupply_possible (true | false)
resources: {fuel, kits, personnel}
missions: [ {id, severity (1-5), distance, deadline, kits, personnel,
             route_blocked, alternate_distance | null} ]
update_notice, update_report      (Round 5 only, from stage 1 onwards)
```

4. Output: the Decision

```
{ "action": "dispatch | reroute | wait | reject | resupply",
  "priority": 1-5,
  "reason": "short explanation",
  "mission_id": "M1",                 (needed if more than one mission)
  "uses_alternate_route": true/false,
  "fuel_needed": n, "kits_needed": n, "personnel_needed": n }
```

Action	Meaning
dispatch	Send a team to the mission by the direct route
reroute	Send a team by the alternate route (direct road is blocked)
wait	Nothing can be done safely right now, but the situation may improve
reject	The mission cannot be completed at all (for example, no route or the deadline cannot be met)
resupply	Request more fuel, kits or personnel because resources are the problem

5. Rules Common to All Rounds

Rule	Detail
Fuel needed	ceil(route distance / 10)
Travel time	ceil(route distance x weather factor). It must be less than or equal to the mission deadline.
Weather factor	clear = 1.0, rain = 1.25, storm = 1.5
Route distance	The direct **distance** if the road is open; the **alternate_distance** if the direct road is blocked.
Blocked road	A blocked direct route must never be used. If alternate_distance is null, there is no route and the mission is infeasible.
Resources	The team needs fuel >= fuel needed, kits >= mission kits and personnel >= mission personnel.
One decision	Each decision concerns exactly one mission.
Safety first	An unsafe dispatch (blocked road, not enough resources or a missed deadline) is never acceptable, whatever the severity.
Choosing between missions	Among feasible missions, prefer the higher severity. Break ties by the tighter deadline, then by lower fuel use.

Feasibility check (used in every round)

```
km     = distance if not route_blocked else alternate_distance   # None -> infeasible
fuel   = ceil(km / 10)
time   = ceil(km * weather_factor)
feasible = (km is not None) and fuel <= resources.fuel
           and mission.kits <= resources.kits
           and mission.personnel <= resources.personnel
           and time <= mission.deadline
```

All numbers in the samples below are illustrative.

Round 1: Basic Rescue (10 points)

Description

A single mission, clear weather and an open road. The agent only has to decide whether the rescue is possible with the available fuel and in the available time.

Input

- One mission; weather is always clear; `route_blocked` is false.
- Resources: fuel, kits, personnel.

Task

- Compute fuel needed and travel time for the mission.
- Dispatch if fuel, kits, personnel and deadline are all satisfied.
- Otherwise reject or wait, and give the reason.

Constraints

- Exactly one mission per case.
- Decision must contain action, priority (1-5) and a reason.

Sample

```
Sample 1
weather: clear | resources: fuel 5, kits 3, personnel 2
M1: severity 4, distance 30, deadline 40, kits 1, personnel 1
fuel = ceil(30/10) = 3 <= 5     time = ceil(30 x 1.0) = 30 <= 40
Output: action = dispatch, priority = 4, mission_id = M1, fuel_needed = 3

Sample 2
weather: clear | resources: fuel 5, kits 3, personnel 2
M1: severity 5, distance 80, deadline 100, kits 1, personnel 1
fuel = ceil(80/10) = 8 > 5      -> not enough fuel
Output: action = reject (or wait), priority = 5, reason = "Insufficient fuel"
```

Round 2: Blocked Routes (15 points)

Description

One or two missions. Some direct roads are blocked. A longer alternate route may exist, but it needs more fuel and more time, and sometimes it does not exist at all.

Input

- 1-2 missions with `route_blocked` and `alternate_distance` (a number or null).
- Weather may be clear or rain.

Task

- Never use a blocked road.
- If blocked and an alternate exists, run the feasibility check on the alternate distance and return reroute (or uses_alternate_route = true).
- If blocked and the alternate is null, the mission is infeasible.
- With two missions, choose the feasible one with higher severity.

Constraints

- Fuel must be calculated from the route actually used (alternate distance for a rerouted mission).
- mission_id is required when there is more than one mission.

Sample

```
Sample 1
weather: clear | resources: fuel 6, kits 2, personnel 2
M1: severity 5, distance 20 (BLOCKED), alternate 35, deadline 40, kits 1, personnel 1
fuel = ceil(35/10) = 4 <= 6     time = 35 <= 40
Output: action = reroute, mission_id = M1, uses_alternate_route = true, fuel_needed = 4
```

```
Sample 2
M1: severity 5, distance 20 (BLOCKED), alternate = null
No usable route.
Output: action = reject, priority = 5, reason = "Route blocked and no alternate"
```

Round 3: Limited Resources (20 points)

Description

Two or three missions, and the team's fuel, kits and personnel are limited. Sometimes no mission can be served with what is available. Resupply may or may not be possible.

Input

- 2-3 missions; limited resources; `resupply_possible` may be true.

Task

- Check every mission against the resources, route and deadline.
- Dispatch (or reroute) the best feasible mission.
- If nothing is feasible: choose **resupply** when resupply is possible and the shortage is only in resources; choose **reject** when the route or deadline makes it impossible anyway; otherwise **wait**.

Constraints

- Never dispatch a mission the resources cannot support, even if its severity is the highest.
- Doing nothing while a feasible rescue exists is a poor decision.

Sample

```
Sample 1
weather: clear | resources: fuel 4, kits 2, personnel 2
M1: severity 5, distance 30, deadline 40, kits 3, personnel 1   -> kits 3 > 2, infeasible
M2: severity 3, distance 25, deadline 40, kits 1, personnel 1   -> fuel 3, time 25: feasible
Output: action = dispatch, mission_id = M2, priority = 3

Sample 2
resupply_possible: true | resources: fuel 1, kits 2, personnel 2
M1: severity 4, distance 30, deadline 40  -> needs fuel 3 > 1, but the deadline is fine
Output: action = resupply, priority = 4, reason = "Fuel shortage, resupply available"
```

Round 4: Critical Priorities (25 points)

Description

Three to five missions with tight deadlines compete for the same limited resources. The most severe mission is not always the one that can be saved, so the agent must balance severity and feasibility.

Input

- 3-5 missions; tight deadlines; limited resources; blocked roads and rain are possible.

Task

- Filter out every infeasible mission first (safety first).
- From the rest, choose the highest severity; break ties by the tighter deadline, then lower fuel use.
- Return dispatch or reroute for that mission, with priority equal to its severity.
- If none is feasible, fall back to resupply / wait / reject as in Round 3.

Constraints

- Exactly one mission is chosen per decision.
- Always output a consistent, deterministic result for the same input.

Sample

```
Sample
weather: clear | resources: fuel 6, kits 4, personnel 3
M1: severity 5, distance 90, deadline 120  -> fuel 9 > 6          infeasible
M2: severity 4, distance 50, deadline 45   -> time 50 > 45        infeasible
M3: severity 4, distance 40, deadline 50   -> fuel 4, time 40     FEASIBLE
M4: severity 2, distance 10, deadline 30   -> fuel 1, time 10     FEASIBLE
Output: action = dispatch, mission_id = M3, priority = 4, fuel_needed = 4
```

The highest severity (M1) is skipped because it cannot be done safely; M3 wins over M4 because it has the higher severity.

Round 5: Adaptive RescueOps (30 points)

Description

The full problem, now changing over time. The case arrives in several stages. After the first stage, the weather may change and a free-text **update report** describes what changed (new weather, blocked roads, resource changes). The fields of the original case become outdated, and the report is the truth.

Input

- Everything from Rounds 1-4, plus `stage`, `update_notice` and `update_report` (free text).
- Example report: "Weather is now storm. Route to M2 is now blocked. Fuel is now 4."

Task

- Read the update report and extract the changes (weather, route status, resources).
- Apply them to the scenario (an LLM can be used to interpret the text, but its result must be checked).
- Decide again on the updated scenario using the Round 4 logic.
- Each stage is a separate call: every decision must reflect the current state only.

Constraints

- Round 1-4 logic must still work inside this round.
- An LLM reading of the report must be double-checked (blocked route, resources, deadline) before it is trusted.
- An invalid or unsafe decision at any stage is not acceptable.

Sample

```
Stage 0 (original case)
weather: clear | resources: fuel 8, kits 3, personnel 2
M1: severity 3, distance 20, deadline 35, kits 1, personnel 1
M2: severity 5, distance 15, alternate 40, deadline 50, kits 1, personnel 1
Best choice: M2 (severity 5, fuel 2, time 15)

Stage 1 update_report:
"Weather is now storm. Route to M2 is now blocked. Fuel is now 4."
Updated state: weather storm (x1.5), fuel 4, M2 direct road blocked
M2 via alternate: time = ceil(40 x 1.5) = 60 > 50   -> infeasible
M1: fuel 2 <= 4, time = ceil(20 x 1.5) = 30 <= 35   -> feasible
Output: action = dispatch, mission_id = M1, priority = 3, fuel_needed = 2
```

The decision changes because the situation changed: the severe mission M2 is no longer safely reachable, so the agent saves M1.